

SEN381 Assignment 3

Research for Quality Security and Production Readiness Decisions

Research preparation for CivicConnect Milestone 3

Belgium Campus iTversity

Bachelor of Computing

Academic year 2026

Team members

Name	Student number
Thaban Ndlozi	602341
Anele Nkayi	577168
Lebogang Masia	601562

Participation statement: All three members participated in this assignment. There were no nonparticipating members.

Purpose

This research examines how software teams gather convincing evidence about quality, security and production readiness. It compares techniques, explains their limits and identifies the decisions that CivicConnect Milestone 3 (M3) must still make.

The main finding is that release confidence comes from complementary evidence linked to real risks. Passing functional tests alone cannot demonstrate correct access control, usable workflows, safe recovery or reliable operation. Research can guide those checks, but the actual requirements, implementation and operating conditions must determine the final choices.

The examples describe a general web-based information system. They are not an approved CivicConnect test strategy, security design, hosting choice or release decision. No test execution, deployment or repository activity is claimed.

Contents

Section	Page
Question 1 Quality strategy and risk-based verification	3
Question 2 Security engineering and threat-based controls	5
Question 3 Production readiness and operational evidence	7
Question 4 Research to M3 engineering map	10
References	11
Appendix A AI Research and Verification Record	13

Research approach and scope

The report follows Questions 1 to 4 in the assignment brief. It uses Harvard in-text citations and a consolidated reference list. The brief requires research to inform later engineering judgement without presenting M3 choices as already made (Belgium Campus iTversity, 2026).

Sources include professional testing guidance, a NIST secure-development framework, an empirical security study, OWASP guidance and operational engineering documentation. Older publications support established principles; live official documentation supports current technical guidance. Sources were checked on 24 September 2026. An undated webpage is referenced as n.d.; its access date is not treated as its publication year.

The comparisons and example decisions are this report's synthesis of the sources. Example priorities are qualitative rather than measured project probabilities. Thresholds such as acceptable latency, downtime and data loss require agreement with stakeholders in M3. No platform is preferred merely because its documentation is cited.

Terms used

Term	Meaning
QA and QC	Quality Assurance and Quality Control
V&V	Verification and Validation
CI	Continuous Integration
SAST and DAST	Static and Dynamic Application Security Testing
SCA	Software Composition Analysis for third-party dependencies
RTO and RPO	Recovery Time Objective and Recovery Point Objective
SLI and SLO	Service Level Indicator and Service Level Objective

Question 1 Quality strategy and risk based verification

1.1 How QA QC and V and V work together

QA improves the processes used to prevent defects, such as review practices and clear acceptance criteria. QC evaluates the product, including through testing. Verification checks conformance to specified requirements; validation checks whether the product serves its intended users and purpose. A technically correct screen may still be confusing. Results should therefore improve both the product and the development process (ISTQB, 2024, sections 1.1-1.3).

1.2 Choosing effort according to risk

Risk-based verification considers the likelihood and impact of failure when selecting depth and priority (ISTQB, 2024, section 5.2). A team can identify important assets and workflows, describe plausible failures, rank them and select evidence capable of exposing each one. Ratings should consider exposure, complexity, recent change, defect history and business criticality. A rare privacy breach can deserve more attention than a frequent cosmetic defect.

Our interpretation is to treat ratings as discussion aids, not precise predictions. For example, a changed permission boundary needs negative access tests even when its normal path is already covered. A low-risk label change may need a focused UI check and relevant regression tests. Revisit priorities after defects or architecture changes; record what remains untested and who can accept that risk.

1.3 Complementary evidence and its limits

Evidence	Strong at exposing	Limitations and cost
Unit tests	Incorrect calculations, boundary conditions and rule branches in a small component.	Fast and easy to diagnose, but mocks can hide integration failures; cannot prove deployment or usability.
Integration and API tests	Broken contracts, database mappings, transactions and access checks across real boundaries.	Need representative dependencies and data; API success does not prove the browser workflow is usable.
System and end-to-end tests	Failures in a complete journey through UI, service and storage.	Slower and more fragile; a few journeys cannot cover all roles, inputs or failure paths.
Static analysis and review	Unsafe patterns, suspicious data flows and design mistakes before execution.	Tool rules have false positives and omissions; review quality depends on expertise. Runtime behaviour remains unproven.
Load and soak tests	Latency under load, saturation and resource growth over time.	Costly setup; conclusions apply only to the workload, duration and environment tested.

Vocke (2018) supports a mix of small tests and fewer broad tests, while warning against rigid interpretations of the test pyramid. The table applies that reasoning to web-system risks. A high test count is less useful than coverage of important failure paths. Regression means repeating relevant checks after change and can include several levels, rather than being a separate test level.

Question 1 Evidence map and quality gates

1.4 Risk to Verification Evidence Map

Figure 1 is an original example map. Each arrow links a reason for concern to evidence that can challenge it, then to a conditional progression decision. These are research examples, not CivicConnect release gates.

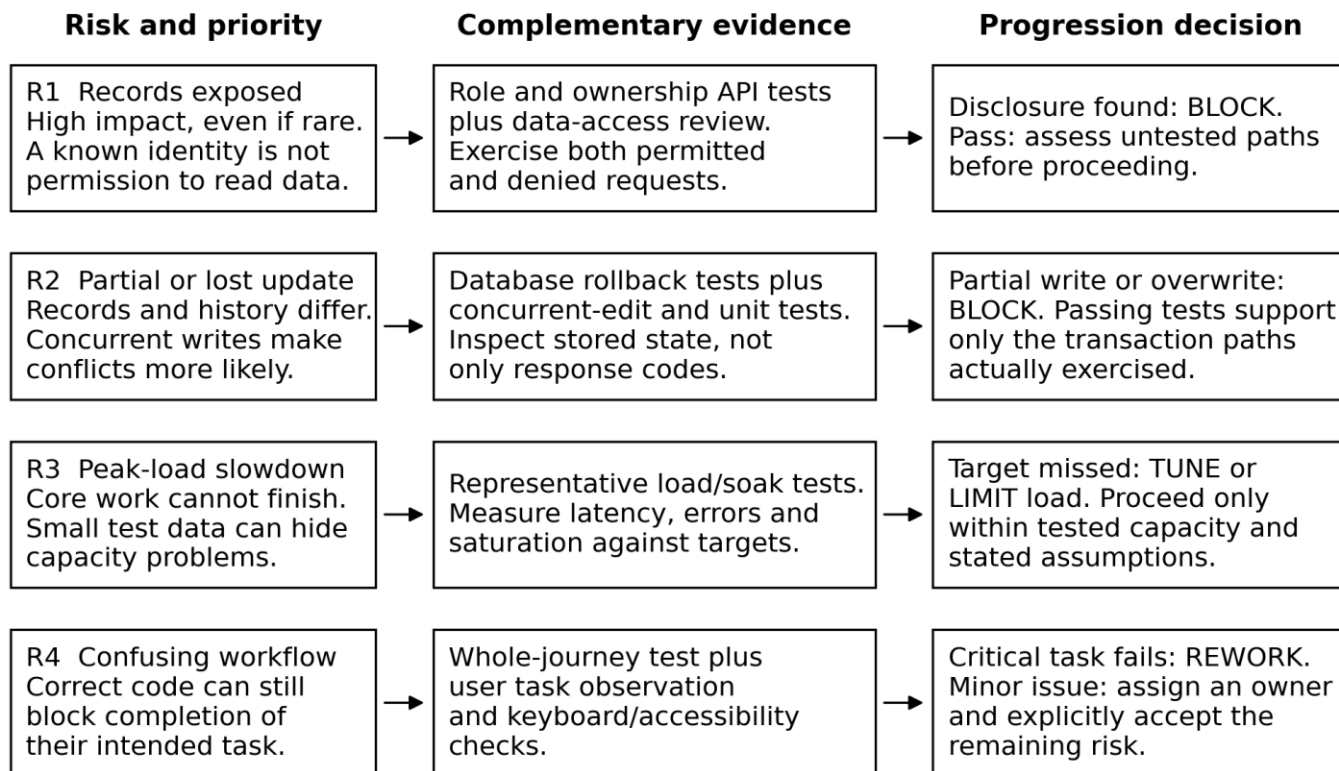


Figure 1. Original synthesis informed by ISTQB (2024), Vocke (2018) and OWASP Foundation (n.d.-b). User observation contributes validation alongside technical verification.

1.5 Automation and proportionate gates

Repeatable checks need versioned test code, known dependencies, controlled data, a clean environment and retained results tied to the exact revision. Fast checks can run on each pull request; slower integration and performance checks can run at an appropriate later gate. Teams must not release a different untested build. Automation improves repeatability but needs maintenance and review (Vocke, 2018; McNutt, 2016).

A defensible candidate gate blocks a failed critical rule, unauthorised access or corrupt write. A style advisory may warn while its usefulness is assessed. An unstable test should be investigated with an owner and deadline; repeatedly rerunning it until green hides uncertainty. If an essential check is skipped, its evidence is missing, even when the pipeline reports success.

1.6 Why passing every automated test is insufficient

Passing tests means the selected assertions held for the exercised inputs and environment. It does not establish that the requirements were right, the assertions were meaningful or omitted conditions are safe (ISTQB, 2024). For example, successful login tests say nothing about access to another user's records. Release judgement also needs threat review, representative-user validation, operational evidence and explicit treatment of residual risks. Automation supports that judgement; it cannot make all of it.

Question 2 Security engineering and threat based controls

2.1 Threat modelling as a decision process

Threat modelling identifies assets, data flows, actors and trust boundaries, then asks how the system could be misused. STRIDE provides six prompts: spoofing, tampering, repudiation, information disclosure, denial of service and elevation of privilege. Its value is the control decisions it prompts, not the number of threats listed (OWASP Foundation, n.d.-e).

For a web system, first sketch the browser, application, identity service, database and deployment pipeline. Mark where trust changes. Apply STRIDE at these boundaries, prioritise plausible harm, translate threats into testable requirements and review the model after change. A changed record identifier, for example, prompts an ownership-check requirement, not simply a stronger login. A model can still miss an omitted service or unexpected abuse; it needs operational feedback.

2.2 Threat to Control Traceability Table

The four concern areas below are authentication, authorisation, secrets/configuration and input/data handling. Controls and tests are candidates. The chains apply OWASP guidance (n.d.-a, n.d.-b, n.d.-c and n.d.-d respectively).

Threat or misuse	Requirement or objective	Possible control	Verification evidence	Residual risk
Authentication: stolen passwords used to take over accounts.	Limit account takeover and abuse of recovery.	Established identity mechanism; risk-appropriate MFA; throttling; protected reset and session handling.	Attempt repeated logins, reset-token reuse and expired-session access; inspect MFA and recovery paths.	Phishing, stolen sessions and compromised recovery channels may remain.
Authorisation: user changes a record ID or calls an administrator endpoint directly.	Every operation must respect role and object ownership.	Server-side checks on every request; deny by default; scope queries to allowed records.	Use two users and different roles; test read, change, delete and list paths, including denied cases.	Missing endpoints, stale permissions or an incorrect policy can still expose data.
Secrets/configuration: a database credential appears in a commit, log or build artifact.	Restrict credential access and allow prompt revocation.	Secret store or protected runtime injection; least privilege; separate environment credentials; rotation.	Scan history and artifacts; inspect log redaction and access policy; test rotation and rejection of old credentials.	Unknown secret formats may evade scans; a compromised authorised workload may still read its secrets.
Input/data: crafted input changes a database query.	Treat input as data and prevent unauthorised query execution.	Parameterised queries; allowlists for dynamic identifiers; limited database permissions.	Review query construction and test malicious values at API/database boundaries.	Untested dynamic queries remain; parameterisation does not itself prevent XSS or business-rule abuse.

Question 2 Complementary controls and remaining uncertainty

2.3 Design construction and checking have different jobs

Secure design establishes trust boundaries and permitted actions. Secure coding implements them, while code and dependency checks look for defects and vulnerable components. These activities complement each other: a parameterised query can still disclose private records if the ownership check is missing.

NIST's Secure Software Development Framework describes practices that integrate security into the development lifecycle, including reducing vulnerabilities and addressing their causes (Souppaya, Scarfone and Dodson, 2022). Applied here, threat findings should become reviewable requirements, design controls and abuse tests before release, with a process to respond to new findings after release. A final penetration test adds evidence but cannot repair a lifecycle that never identified sensitive assets.

2.4 Comparing security checks

Technique	Useful evidence	What still needs complementary work
SAST	Finds suspicious code patterns or data flows without executing the system.	Depends on language support and rules; may miss contextual access rules or produce false alarms.
DAST	Exercises a running application and observes exposed behaviour and some configuration weaknesses.	Limited by reachable routes, accounts and payloads; often cannot infer the intended permission policy.
SCA	Matches discovered third-party versions against known vulnerability information.	An absent advisory is not proof of safety; incomplete inventories and unknown flaws remain.
Human threat and code review	Can question trust assumptions, recovery paths and business misuse.	Takes time and expertise; reviewers also miss defects, so retain automated and runtime evidence.

OWASP supports maintaining an inventory, reviewing vulnerabilities and responding to affected components (OWASP Foundation, n.d.-f). Assess exposure, apply supported fixes and retest. Severity helps prioritise investigation but does not alone describe business impact.

2.5 Limits of automated tools and AI advice

First, coverage is incomplete: a scanner may lack credentials, framework support or deployed configuration. Second, tools struggle with intent: approving one's own request may be valid code but an invalid business rule. Investigate important untested paths and confirm findings with evidence.

Third, AI advice can omit context, suggest outdated settings or sound more certain than the evidence allows. Perry et al. (2023) found less secure code and greater confidence among assisted participants in their study. The work used an older assistant and specific tasks, so it does not measure every current model. It does support independently checking generated controls against authoritative guidance and adversarial tests.

2.6 What a clean scanner report leaves unproven

No high-severity findings does not prove correct ownership checks, secure account recovery, minimal data collection, safe secret access, adequate audit records or resistance to unknown vulnerabilities. It also does not prove that the scanned revision and environment match production. The report is bounded evidence about particular checks; the remaining claims need threat-linked tests, review and operational observation.

Question 3 Environments and controlled deployment

3.1 Environment strategy and configuration

Development supports rapid local work; test or staging supports repeatable verification; production serves real users and data. Differences in runtime versions, schema, permissions or backing services can invalidate testing. Twelve-Factor guidance recommends reducing avoidable differences between development and production (Twelve-Factor App, n.d.). Similarity should focus on behaviour, not copying private production data into less protected environments.

A small staging environment costs less but cannot establish full production capacity. A highly similar staging environment increases confidence but costs more to maintain. A proportionate approach would document material differences, reproduce security and dependency behaviour, and use synthetic or suitably protected representative data. Environment parity is a controlled approximation, not a guarantee.

Keep configuration definitions and non-sensitive defaults versioned, with environment-specific values supplied separately. Production credentials need restricted access, rotation and separation from test credentials (OWASP Foundation, n.d.-c). Environment variables are a delivery mechanism, not a complete secret-management policy: permissions, diagnostic output and process compromise still matter. Validate missing or unsafe settings before accepting traffic.

3.2 From a successful build to a usable release

A build produces an artifact; deployment places that artifact into an environment with configuration and dependencies; release makes the capability available to users. A repeatable process should identify the source revision, artifact and configuration, record approvals, deploy consistently and verify behaviour afterwards (McNutt, 2016). A process can start successfully while authentication, migrations or external connectivity remain broken.

Deployment option	Benefit	Cost or limitation
Recreate or planned maintenance	Simple sequence and lower infrastructure cost.	Service interruption may be unacceptable; failed startup extends downtime.
Rolling replacement	Updates instances gradually and limits interruption.	Old and new versions coexist, requiring compatible data and interfaces.
Blue-green or canary	Blue-green supports switching environments; canary limits initial exposure and compares signals.	Extra capacity, traffic routing and reliable health signals are needed; shared database changes may prevent reversal.

These options are research comparisons, not platform selections. The release decision should depend on tolerated interruption, team support capacity, data compatibility and the strength of detection. A small system with a permitted maintenance window may gain little from complex traffic splitting.

3.3 Candidate release evidence

Useful evidence includes the exact artifact identifier, deployment log, validated configuration, migration rehearsal, critical-path smoke checks and a named person responsible for stopping the release. Smoke checks should verify a useful operation, not only an HTTP health response. Approval criteria and the observation period must be defined before exposure, so a failed release is not accepted merely because deploying it took effort.

Question 3 Recovery observability and operational quality

3.4 Failure and recovery

Rolling application code back does not automatically restore database compatibility. An additive change can retain the old field or interface while consumers migrate, removing it only later. Sato (2014) describes this expand-migrate-contract approach. It improves reversibility but requires extra transition work. A destructive migration may need a forward repair or controlled restore rather than a blind code rollback.

RTO is the target time to restore service; RPO is the acceptable age of the recovery point, expressing potential data loss. Backups need protected storage and actual restoration exercises; a successful backup job is not proof that recovery will meet either objective (Amazon Web Services, n.d.). Restoring an older backup may discard valid recent work, so reconciliation and stakeholder communication matter.

If a release fails, a candidate response is to stop further rollout, limit harm, notify the responsible person and preserve diagnostic evidence without exposing sensitive data. Choose rollback, forward repair or recovery according to data compatibility, then validate core operations and data integrity. Record the incident and improve the checks that failed to detect it. M3 must define authority and acceptable recovery targets.

3.5 Signals must lead to action

Logs record events useful for diagnosis; metrics describe trends and rates; traces connect steps across a request path (OpenTelemetry Authors, n.d.). Monitoring evaluates these signals against expected behaviour; an alert calls attention to a condition needing action. Google's SRE guidance emphasises latency, traffic, errors and saturation, with alerts that deserve a response (Ewaschuk, 2016).

For example, increased response time with growing database saturation can suggest a bottleneck. Error rate and a user-facing synthetic check help distinguish actual user harm from harmless resource spikes. An alert needs a severity, owner and response instruction. Collecting more logs without retention limits or usable queries increases cost and privacy exposure without necessarily improving diagnosis.

3.6 Operational trade-offs

Quality concern	Evidence and decision trade-off
Reliability	Define user-facing availability and failure targets. Redundancy can reduce some outages but adds cost and failure modes.
Performance	Measure latency percentiles and errors under representative load; averages can hide slow requests. Faster service may need optimisation or more resources.
Scalability	Test growth and saturation. Adding application instances may not solve database contention; scaling limits must be known.
Cost	Estimate and monitor compute, storage, traffic, backup and telemetry costs. Extra capacity cannot be assumed affordable.
Supportability	Rehearse diagnosis and recovery from written instructions. A design dependent on one person may be hard to operate safely.

Service objectives should express user needs and guide trade-offs rather than demand perfect availability at any cost (Beyer et al., 2016, chapter 4). The table applies this reasoning; numerical targets must come from the actual service context.

Question 3 Production Readiness Evidence Matrix

3.7 Evidence before and after release

This matrix combines the preceding research into candidate evidence. A pre-release check reduces uncertainty; continued observation tests whether its assumptions hold in operation. It contains eight concerns and does not certify a particular product as ready.

Concern	Risk if ignored	Evidence before release	Evidence after release
Environment and configuration	Unexpected runtime, permissions or service differences break behaviour.	Version/configuration inventory; staging checks; explicit list of material differences.	Configuration drift reports, runtime versions and dependency health.
Secrets and access	Leaked or excessive credentials expose production data.	Access-policy review; environment separation; tested rotation and safe logs.	Secret-access audit, expiry/rotation status and suspicious use.
Release integrity	Wrong or unverified artifact is deployed.	Artifact/revision identity, approved changes, deployment rehearsal and smoke results.	Deployment record, active version and critical-journey success.
Data changes and rollback	Old code cannot read new data; rollback worsens harm.	Migration compatibility tests; preserved rollback artifact; recovery decision and rehearsal.	Migration errors, integrity checks and old/new-version compatibility.
Backup and recovery	Outage or data loss lasts beyond acceptable limits.	Successful restore exercise; measured recovery time and recovery-point age against agreed targets.	Backup age/failures and repeated restore exercises; actual incident recovery times.
Observability and response	Failures remain undetected or alerts are ignored.	Known-failure exercise proving logs, signals, alert routing and response instructions.	User-facing errors and latency, alert usefulness and response time.
Capacity and cost	Peak demand causes failure or unaffordable spending.	Representative load/soak results, capacity limits and cost estimate.	Traffic, latency percentiles, saturation and spending versus expectations.
Support and ownership	No one can diagnose, approve recovery or communicate an incident.	Named responsibilities, runbook rehearsal, escalation and known-risk record.	Incident history, unresolved actions, support demand and runbook accuracy.

Source synthesis: Twelve-Factor App (n.d.); McNutt (2016); Sato (2014); Amazon Web Services (n.d.); Ewaschuk (2016); OWASP Foundation (n.d.-c).

3.8 Why local success and functional tests are insufficient

A developer's machine may use different credentials, data volumes, versions and network conditions. Functional tests can pass while migration, capacity, recovery or alert routing remains untested. Production readiness therefore requires evidence that the product can serve users, fail safely and be supported within agreed constraints. A cloud deployment proves none of those claims by itself.

Question 4 Research to M3 Engineering Map

The following eight findings span quality, security and production readiness. Each final column leaves a concrete CivicConnect decision open for M3, using its actual requirements, architecture, implementation and evidence.

Research finding	Engineering concern	Candidate approach or evidence	Trade-off or limitation	Decision M3 must still make
Risk drives verification depth (ISTQB, 2024).	Critical failures receive too little attention.	Rank product risks and link them to test/review evidence.	Ratings rely on incomplete information.	Which workflows and failures are most critical, and what depth is justified?
Different test levels expose different defects (Vocke, 2018).	Integration defects survive isolated tests.	Unit rules plus real-dependency API tests and selected whole journeys.	Broader tests cost more and can be unstable.	Which boundaries require real integration tests and which journeys need end-to-end checks?
Threat models connect misuse to controls (OWASP Foundation, n.d.-e).	Hidden trust assumptions.	STRIDE review of data flows and threat-control-evidence links.	Model quality depends on scope and reviewer knowledge.	What are the actual assets, trust boundaries and highest-priority threats?
Authentication does not establish permission (OWASP Foundation, n.d.-b).	Cross-user or privileged access.	Object- and role-based negative access tests with server policy review.	Tests need an agreed policy and representative identities.	Which roles, ownership rules and operations must be enforced and verified?
Security scans provide bounded evidence (OWASP Foundation, n.d.-f).	Missed library risks.	Dependency inventory, advisory review and complementary abuse tests.	Unknown flaws and contextual misuse can remain.	Which scanners, review gates and risk-acceptance responsibilities fit the stack?
Repeatability and parity reduce deployment uncertainty (McNutt, 2016).	Untested environment or wrong artifact.	Versioned deployment process, staging comparison and smoke checks.	Parity and extra environments cost time and money.	Which environment, platform and deployment approach meet project constraints?
Recovery must include data (Amazon Web Services, n.d.; Sato, 2014).	Failed recovery or lost data.	Migration compatibility and backup restoration rehearsal.	Restore may lose recent work; compatibility requires extra design.	What RTO/RPO and rollback or repair procedure are acceptable?
Signals need objectives and owners (Ewaschuk, 2016).	Undetected degradation or ignored alerts.	Latency/error metrics, diagnostic logs and rehearsed alert response.	Noise, storage cost and sensitive telemetry require control.	Which SLOs, signals, alert thresholds and responders are justified?

The research supports asking for linked evidence and explicit trade-offs. It does not select CivicConnect tools, controls, deployment services or numerical release thresholds. M3 must record and defend those choices after applying the research to the project.

References

- Amazon Web Services (n.d.) Back up data. AWS Well-Architected Framework, Reliability Pillar. Available at: <https://docs.aws.amazon.com/wellarchitected/latest/reliability-pillar/back-up-data.html> (Accessed: 24 September 2026).
- Belgium Campus iTversity (2026) SEN381 Assignment 3 Research for Quality, Security and Production Readiness Decisions. Unpublished assignment brief.
- Beyer, B., Jones, C., Petoff, J. and Murphy, N.R. (eds.) (2016) Site Reliability Engineering: How Google Runs Production Systems. Sebastopol, CA: O'Reilly Media. Chapter 4, Service Level Objectives. Available at: <https://sre.google/sre-book/service-level-objectives/> (Accessed: 24 September 2026).
- Ewaschuk, R. (2016) Monitoring Distributed Systems. In: Beyer, B. et al. (eds.) Site Reliability Engineering. Sebastopol, CA: O'Reilly Media, chapter 6. Available at: <https://sre.google/sre-book/monitoring-distributed-systems/> (Accessed: 24 September 2026).
- ISTQB (2024) Certified Tester Foundation Level Syllabus. Version 4.0.1, 15 September. International Software Testing Qualifications Board. Available at: https://istqb.org/wp-content/uploads/2024/11/ISTQB_CTFL_Syllabus_v4.0.1.pdf (Accessed: 24 September 2026).
- McNutt, D. (2016) Release Engineering. In: Beyer, B. et al. (eds.) Site Reliability Engineering. Sebastopol, CA: O'Reilly Media, chapter 8. Available at: <https://sre.google/sre-book/release-engineering/> (Accessed: 24 September 2026).
- OpenTelemetry Authors (n.d.) Signals. OpenTelemetry Documentation. Available at: <https://opentelemetry.io/docs/concepts/signals/> (Accessed: 24 September 2026).
- OWASP Foundation (n.d.-a) Authentication Cheat Sheet. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html (Accessed: 24 September 2026).
- OWASP Foundation (n.d.-b) Authorization Cheat Sheet. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html (Accessed: 24 September 2026).
- OWASP Foundation (n.d.-c) Secrets Management Cheat Sheet. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html (Accessed: 24 September 2026).
- OWASP Foundation (n.d.-d) SQL Injection Prevention Cheat Sheet. Available at: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html (Accessed: 24 September 2026).
- OWASP Foundation (n.d.-e) Threat Modeling Cheat Sheet. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Threat_Modeling_Cheat_Sheet.html (Accessed: 24 September 2026).
- OWASP Foundation (n.d.-f) Vulnerable Dependency Management Cheat Sheet. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Vulnerable_Dependency_Management_Cheat_Sheet.html (Accessed: 24 September 2026).
- Perry, N., Srivastava, M., Kumar, D. and Boneh, D. (2023) Do Users Write More Insecure Code with AI Assistants? Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, pp. 2785-2799. Author manuscript, version 3. Available at: <https://arxiv.org/abs/2211.03622v3> (Accessed: 24 September 2026).
- Sato, D. (2014) Parallel Change. MartinFowler.com, 13 May. Available at: <https://martinfowler.com/bliki/ParallelChange.html> (Accessed: 24 September 2026).
- Souppaya, M., Scarfone, K. and Dodson, D. (2022) Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities. NIST SP 800-218. Available at: <https://csrc.nist.gov/pubs/sp/800/218/final> (Accessed: 24 September 2026).

Twelve-Factor App (n.d.) X. Dev/prod parity. Available at: <https://12factor.net/dev-prod-parity> (Accessed: 24 September 2026).

Vocke, H. (2018) The Practical Test Pyramid. MartinFowler.com, 26 February. Available at: <https://martinfowler.com/articles/practical-test-pyramid.html> (Accessed: 24 September 2026).

Appendix A AI Research and Verification Record

All three members used the ChatGPT-assisted draft and personally checked its explanations against the cited sources.

Team member	AI tool and use	Purpose	Output used	How independently verified	Changed or rejected
Thaban Ndlozi; Anele Nkayi; Lebogang Masia	ChatGPT; shared draft and document preparation.	Support the research explanations, tables and document layout.	Research text, tables and original figure.	Each member compared the explanations with the cited sources and reviewed the report against the assignment questions.	The final text keeps M3 decisions open and states the limits of testing and security scans.